329

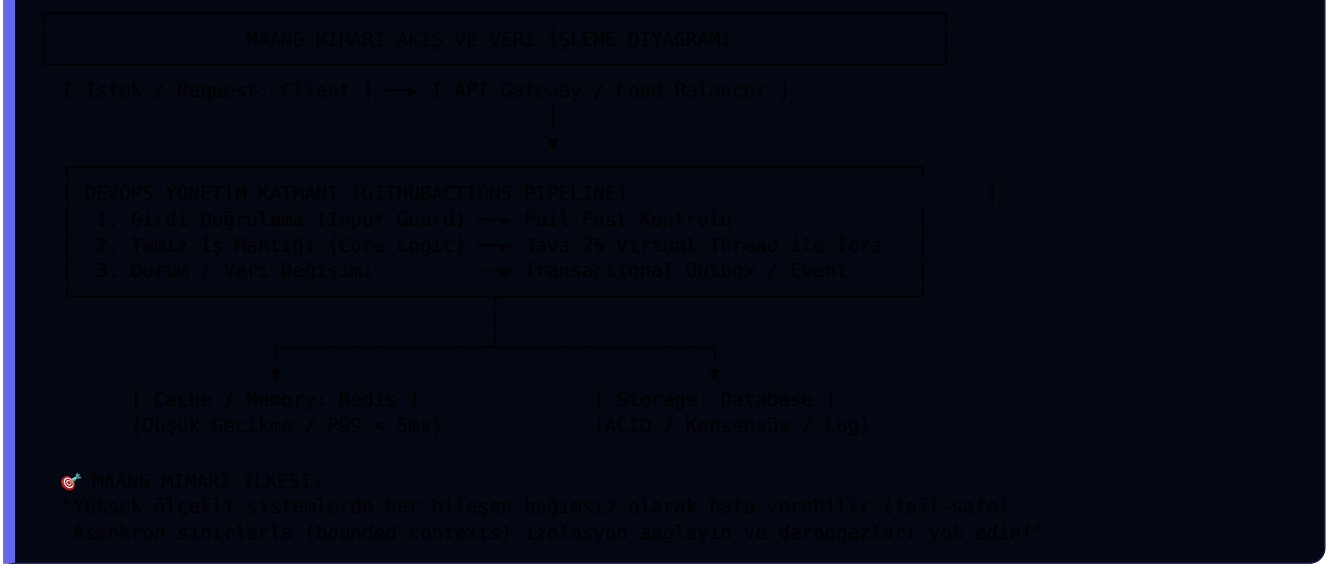
BÖLÜM: CI/CD BORU HATLARI & DEVOPS: GITOPS, GITHUB ACTIONS & SÜRÜM YÖNETIMI | MODULE: CI/CD PIPELINES & DEVOPS: GITOPS, GITHUB ACTIONS & RELEASES

CI/CD Boru Hatları & DevOps: GitOps, GitHub Actions & Sürüm Yönetimi - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

CI/CD Pipelines & DevOps: GitOps, GitHub Actions & Releases - Observability, Telemetry, Tracing & Production Troubleshooting

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (DevOps mikro-servisleri) ve otomatik yönlendirme robotları (GitHubActions akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (DevOps microservices), and dynamic routing switches (GitHubActions pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde CI/CD Boru Hatları & DevOps: GitOps, GitHub Actions & Sürüm Yönetimi Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde DevOps mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): GitHubActions katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for CI/CD Pipelines & DevOps: GitOps, GitHub Actions & Releases: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), DevOps implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in GitHubActions don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: DevOps & GitHubActions Kurumsal Uygulaması
package com.maang.se.module33.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class DevOpsArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record DevOpsRequest(String transactionId, String payload, Instant timestamp) {
        public DevOpsRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record DevOpsResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class DevOpsExecutionService {
        public DevOpsResult processRequest(DevOpsRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new DevOpsResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering DevOps Engine Başlatıldı...");
        var service = new DevOpsExecutionService();
        var req = new DevOpsRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

DevOps kurgulanırken GitHubActions bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling DevOps logic directly to synchronous blocking calls into GitHubActions is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• DevOps mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design DevOps components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.